

BlockQuiz
Web-Based Block Programming Platform with Auto-Grading for
students

Fabio Plunser



B.Sc. Thesis

Supervisor: Ass. Prof. Dr. Michael Vierhauser
2nd January 2026

Abstract

This thesis presents the design and implementation of BlockQuiz, a web-based block programming learning platform with automated grading capabilities, specifically designed for children aged 8-12 years. As computational thinking becomes increasingly integrated into educational curricula, notably through Austria’s mandatory “Digitale Grundbildung” subject introduced in 2022, there is a growing need for accessible, teacher-friendly programming tools that provide immediate feedback without requiring extensive technical expertise from educators.

Unlike existing platforms such as Scratch, Tynker, or Microsoft MakeCode, which tend toward open-ended creative projects or hardware-specific applications, BlockQuiz focuses on short, tightly constrained programming puzzles with clear learning objectives. The platform implements two primary exercise types: Input/Output puzzles for algorithmic thinking and 2D Graphics exercises for visual reasoning and geometry concepts. Each exercise features a constrained block toolbox, reducing cognitive load while guiding learners toward solutions.

The technical implementation leverages modern web technologies including SvelteKit, Google Blockly for the visual programming interface, and a privacy-first architecture enabling self-hosted deployment via Docker. Key contributions include a secure client-side code execution sandbox with timeout protection, a flexible auto-grading system supporting multiple validation strategies, and a teacher-oriented content management system for exercise authoring without programming knowledge.

The platform supports internationalization (German and English), implements progressive hint systems, and captures learning analytics for educational research. An evaluation with target users demonstrates the platform’s usability and effectiveness in teaching core programming concepts including sequences, loops, conditionals, and simple functions.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Ass. Prof. Dr. Michael Vierhauser, for his invaluable guidance, constructive feedback, and continuous support throughout this project. His expertise in software quality and educational technology significantly shaped the direction of this work.

I am also grateful to the Quality Engineering research group at the University of Innsbruck for providing an inspiring academic environment and the resources necessary to complete this thesis.

Special thanks go to all participants who took part in the usability evaluation, providing essential feedback that helped improve the platform's design and functionality.

Finally, I would like to thank my family and friends for their encouragement and patience during the development of this project.

Contents

Abstract	i
Acknowledgments	iii
Contents	v
List of Figures	vii
List of Tables	ix
Declaration	xi
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.2 Goals and Contributions	1
1.3 Target Audience	2
1.4 Scope and Limitations	2
1.5 Document Structure	2
Bibliography	5

List of Figures

List of Tables

Declaration

By my own signature I declare that I produced this work as the sole author, working independently, and that I did not use any sources and aids other than those referenced in the text. All passages borrowed from external sources, verbatim or by content, are explicitly identified as such.

Signed: Date:

1 Introduction

Computational thinking has emerged as a fundamental skill for the 21st century, increasingly recognized as essential for all learners regardless of their future career paths. In Austria, this recognition materialized with the introduction of “Digitale Grundbildung” (Digital Basic Education) as a mandatory subject for secondary education in 2022/23, reflecting a broader European trend toward integrating computational concepts into primary and secondary curricula.

1.1 Motivation and Problem Statement

While numerous block-based programming environments exist, including Scratch, Tynker, Microsoft MakeCode, and Snap!, these platforms present several challenges when deployed in classroom settings with younger learners:

- **Open-ended complexity:** Platforms like Scratch emphasize creative, open-ended projects that can overwhelm beginners and make assessment difficult.
- **Hardware dependencies:** Tools such as MakeCode are often tied to specific hardware (micro:bit, Arduino), limiting accessibility.
- **Lack of automated feedback:** Most existing solutions require manual teacher intervention for assessment, creating bottlenecks in larger classrooms.
- **Privacy concerns:** Cloud-based platforms raise data protection issues, particularly with younger users in European educational contexts.

Teachers, especially those without programming backgrounds, need tools that allow them to create and manage exercises without technical expertise while providing students with immediate, meaningful feedback on their solutions.

1.2 Goals and Contributions

This thesis addresses these challenges through the design and implementation of BlockQuiz, a web-based block programming learning platform. The primary contributions are:

1. **Constrained exercise model:** A framework for creating focused programming puzzles with per-exercise block toolboxes, reducing cognitive load and enabling clear learning objectives.
2. **Client-side auto-grading:** A secure sandbox execution environment with multiple grading strategies for I/O and graphical (Turtle) exercises, providing immediate feedback without server-side code execution.

3. **Teacher-oriented authoring:** A content management system enabling non-programmers to create, edit, and organize exercises and courses.
4. **Privacy-first architecture:** A self-hostable solution deployable via Docker in school networks, with optional guest mode requiring no personal data.
5. **Internationalization:** Full support for German and English, with extensible localization infrastructure.

1.3 Target Audience

The platform primarily targets children aged 8–12 years, a developmental stage where learners can handle block-based logic, loops, and conditionals while benefiting from visual, immediate feedback. Secondary users include teachers who author and manage exercises, and administrators who deploy and maintain the platform.

1.4 Scope and Limitations

The minimum viable product (MVP) encompasses:

- Two exercise types: I/O puzzles and Turtle Graphics
- Ten curated exercises covering sequences, loops, conditionals, and simple functions
- A minimal CMS for exercise and course management
- Client-side sandboxed execution with configurable timeouts
- German and English localization
- Docker-based deployment

Explicitly out of scope are: full classroom management with detailed analytics dashboards, complex simulators beyond 2D canvas/turtle, and integration with institutional single sign-on systems (SAML/SSO), though the architecture supports future extensions.

1.5 Document Structure

The remainder of this thesis is organized as follows:

- **Chapter 2: Background and Related Work** reviews existing block-based programming platforms, educational research on computational thinking, and technical foundations including Google Blockly.
- **Chapter 3: Requirements Analysis** details functional and non-functional requirements derived from educational and technical constraints.
- **Chapter 4: System Design** presents the architecture, including the exercise model, grading framework, and CMS design.
- **Chapter 5: Implementation** describes the technical realization using SvelteKit, Blockly, and the sandbox execution environment.

- **Chapter 6: Evaluation** reports on usability testing with target users and discusses results.
- **Chapter 7: Conclusion** summarizes contributions and outlines future work.

Bibliography